

FULL STACK

WEB DEVELOPMENT



Ajeet Yadav

CHEATSHEET - PAGE 1

1. HTML

HTML (HyperText Markup Language) is used to create the structure of a webpage.

★ What is HTML?

- HTML stands for HyperText Markup Language.
- It is the standard language to create web pages.
- HTML describes the structure of a webpage.
- HTML elements are the building blocks of a webpage.
- It works with CSS (for styling) and JavaScript (for behavior).

★ Basic HTML Document Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Page Title</title>
</head>
<body>
  <!-- Visible content goes here -->
</body>
</html>
```

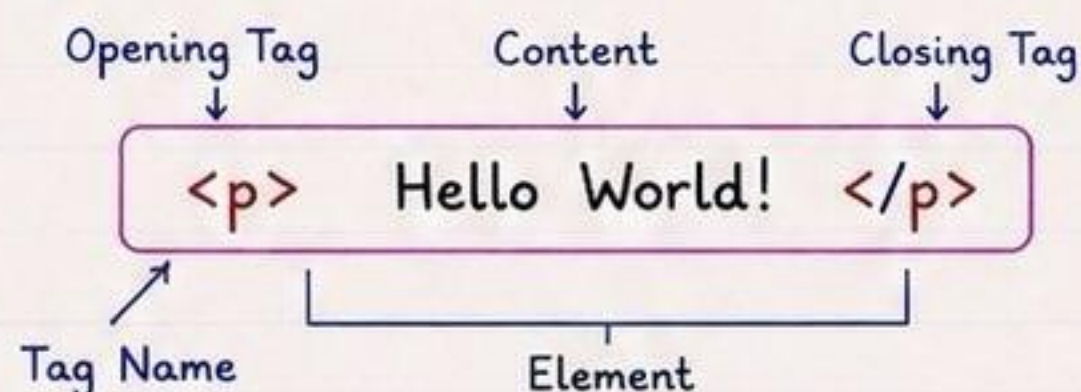
★ Common HTML Tags

Tag	Description	Example
<h1> to <h6>	Headings	Heading 1
<p>	Paragraph	This is a paragraph.
<a>	Link	Google
	Image	
/	Unordered / Ordered List	Item
<div>	Division / Container	Content
	Inline Container	Text
<form>	Form	<form>...</form>
<input>	Input Field	<input type="text">
<button>	Button	<button>Click Me</button>

★ Attributes Example

- Google
→ Opens link in new tab
-
→ Image with size and alternative text
- <input type="email" placeholder="Enter email" required>
→ Email input field (required)
- <button type="submit">Submit</button>
→ Submit button in form

★ HTML Elements



★ Void Elements (No Closing Tag)

These elements do not have a closing tag.

Examples:

```

<br>   <hr>   <input>   <meta>   <link>
```



Remember :

- HTML creates the structure of the webpage.
- Use semantic tags (like <header>, <nav>, <main>, <footer>) for better SEO and readability.
- Always validate your HTML code.

Tip :

Write clean, organized HTML for better maintenance!



2. CSS

CSS (Cascading Style Sheets) is used to style and layout the webpage.

★ What is CSS ?

- CSS stands for Cascading Style Sheets.
- It controls the look and feel of HTML elements.
- It separates content (HTML) from design.
- It makes websites beautiful and responsive.
- It follows a cascading order (priority).

★ Ways to Add CSS

1. Inline CSS - Using style attribute inside an element.

```
<p style="color: red;">This is red text.</p>
```

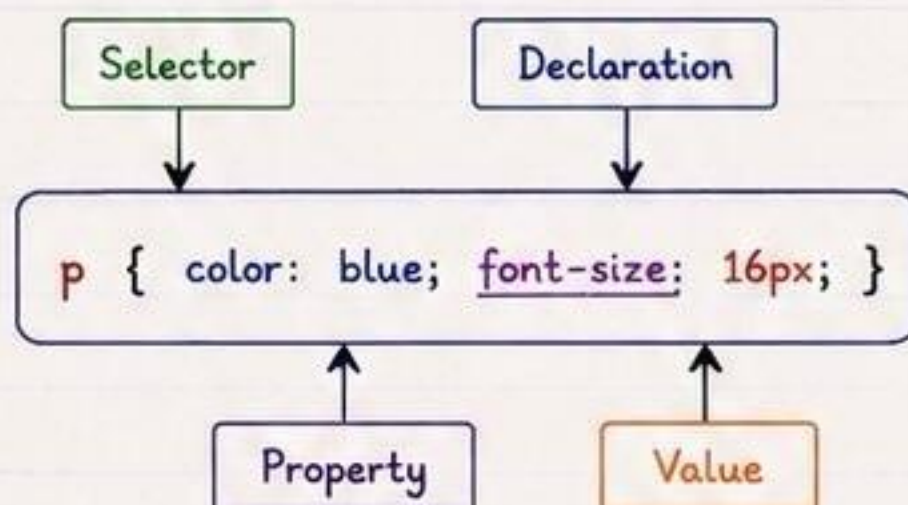
2. Internal CSS - Using <style> tag in <head>.

```
<style>
p { color: blue; }
</style>
```

3. External CSS - Using separate .css file.

```
<link rel="stylesheet" href="style.css">
```

★ CSS Syntax



- **Selector**: selects the HTML element.
- **Property**: what you want to change.
- **Value**: the value of the property.
- **Declaration**: property and value together.

★ CSS Selectors

Selector	Example	Meaning
Element Selector	<code>p { ... }</code>	Selects all <p> elements
Class Selector	<code>.box { ... }</code>	Selects all elements with class="box"
ID Selector	<code>#main { ... }</code>	Selects element with id="main"
Universal Selector	<code>* { ... }</code>	Selects all elements
Group Selector	<code>h1, p { ... }</code>	Selects all <h1> and <p> elements
Descendant Selector	<code>div p { ... }</code>	Selects <p> inside <div>
Child Selector	<code>ul > li { ... }</code>	Selects direct child of
Adjacent Sibling	<code>h1 + p { ... }</code>	Selects <p> next to <h1>

★ CSS Comments

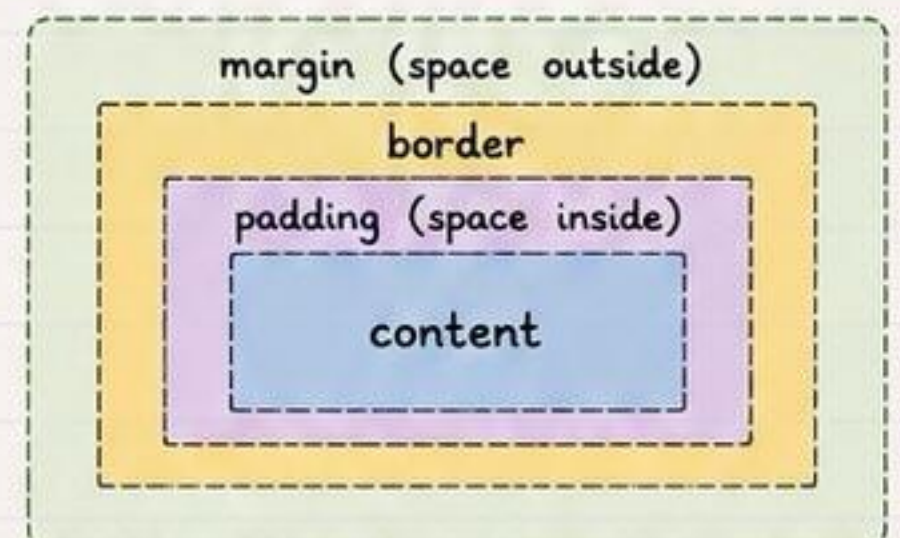
```
/* This is a
   multi-line comment */

/* Single line comment */
```

★ CSS Colors

- By Color Name: red, blue, green
- By Hex: #ff0000, #00ff00, #0000ff
- By RGB: rgb(255, 0, 0)
- By HSL: hsl(0, 100%, 50%)
- By RGBA: rgba(0, 0, 255, 0.5)
(last value is opacity 0 to 1)

★ CSS Box Model



★ Example (in CSS)

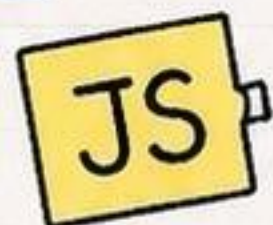
```
p {
  color: darkblue;
  font-size: 18px;
  background-color: #f0f0f0;
  padding: 10px;
  border: 1px solid #ccc;
  margin: 20px;
  text-align: center;
}
```

This is a paragraph styled with CSS.

★ Remember



- CSS controls presentation, HTML controls structure.
- Use external CSS for better performance.
- Keep your CSS clean and organized!



3. JAVASCRIPT

JavaScript is used to add interactivity and logic to the webpage.

★ What is JavaScript?

- JavaScript is a programming language of the web.
- It makes webpages interactive and dynamic.
- It can change HTML and CSS.
- It runs in the browser.

★ JavaScript Syntax

Declaration

Statement

Comments

```
let x = 10;    console.log(x);    // This is a comment
```

- Declaration - Creating variables.
- Statement - Instructs to perform an action.
- Comments - Notes for code (ignored by browser).

★ Data Types

Type	Description	Example
String	Textual data	"Hello"
Number	Numeric data	42, 3.14
Boolean	True or False	true, false
Undefined	Declared but not assigned	let a;
Null	Intentional absence of value	let a = null;
Object	Collection of key-value pairs	{ name: "Om" }
Array	Ordered list of values	[1, 2, 3]

★ Control Statements

1. if - else

```
if (x > 10) {  
  console.log("x is greater");  
} else {  
  console.log("x is smaller");  
}
```

2. for Loop

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

3. while Loop

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

★ Functions

```
function greet(name) {  
  return "Hello, " + name;  
}  
  
let msg = greet("Om");  
console.log(msg); // Hello, Om
```



Remember :

- JavaScript adds behavior to webpages.
- Use meaningful variable names.
- Keep your code clean and well organized.
- Test your code in browser console (F12).

★ Ways to Add JavaScript

1. Internal JavaScript - Using `<script>` in `<head>` or `<body>`.

```
<script>  
  console.log("Hello, JS!");  
</script>
```
2. External JavaScript - Using external .js file.

```
<script src="script.js"></script>
```
3. Inline JavaScript - Using in HTML tag.

```
<button onclick="alert('Hi!')">Click Me</button>
```

★ Variables in JavaScript

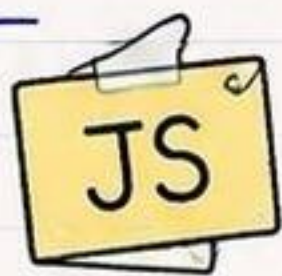
Keyword	Scope	Can Re-declare	Can Re-assign	Example
var	Function Scoped	Yes	Yes	var a = 5;
let	Block Scoped	No	Yes	let b = 10;
const	Block Scoped	No	No	const c = 20;

★ Operators

Arithmetic	+ - * / % **	(add, sub, mul, div, mod, power)
Assignment	= += -= *= /= %= **=	(assign values)
Comparison	== === != !== > < >= <=	(compare values)
Logical	&& !	(and, or, not)
Ternary	condition ? x : y	(if-else shortcut)

Mini Example (HTML + JS)

```
<button onclick="sayHi()">Say Hi</button>  
<script>  
  function sayHi() {  
    alert("Hello from JavaScript!");  
  }  
</script>
```



3. JAVASCRIPT (Contd.)

★ DOM (Document Object Model)

- JavaScript can access and manipulate HTML elements using DOM.
- Every HTML element is a node in a tree structure.
- JavaScript can change content, style, attributes and more.

★ Selecting Elements

Method	Example	Selects
<code>getElementById()</code>	<code>document.getElementById("id")</code>	Element with specific id
<code>getElementsByClassName()</code>	<code>document.getElementsByClassName("box")</code>	All elements with class name
<code>getElementsByTagName()</code>	<code>document.getElementsByTagName("p")</code>	All elements with tag name
<code>querySelector()</code>	<code>document.querySelector(".box")</code>	First element matching CSS selector
<code>querySelectorAll()</code>	<code>document.querySelectorAll(".box")</code>	All elements matching CSS selector

★ Changing Content & Style

- Changing Content**
`element.innerHTML = "<b>Hello</b>";`
`element.textContent = "Hello";`
`element.innerText = "Hello";`
- Changing Style**
`element.style.color = "red";`
`element.style.fontSize = "20px";`
`element.style.backgroundColor = "yellow";`

★ Events

- Events are actions or occurrences that happen in the browser.
- JavaScript can "listen" and "react" to these events.
- Common Events:

<code>click</code>	Mouse click
<code>dblclick</code>	Double click
<code>mouseover</code>	Mouse over element
<code>mouseout</code>	Mouse out of element
<code>keydown</code>	Key is pressed
<code>submit</code>	Form submitted
<code>change</code>	Value changed
<code>load</code>	Page or resource loaded

★ Event Example

```
<button id="btn">Click Me</button>
<p id="msg"></p>
<script>
  document.getElementById("btn").addEventListener("click", function() {
    document.getElementById("msg").innerText = "Button Clicked!";
  });
</script>
```

★ Array in JavaScript

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // Apple
fruits.push("Orange"); // Add at end
fruits.pop(); // Remove last
console.log(fruits.length); // Length of array
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

★ Conditional Statements

```
if (condition) {
  // code
} else if (condition) {
  // code
} else {
  // code
}
```

★ Switch Statement

```
switch (variable) {
  case value1:
    // code
    break;
  case value2:
    // code
    break;
  default:
    // code
}
```

★ Ternary Operator

```
condition ? expressionIfTrue : expressionIfFalse;
let age = 18;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result);
```

★ Template Literals (ES6)

```
let name = "Om";
let msg = `Hello, ${name}!`;
console.log(msg); // Hello, Om!
```



Remember :

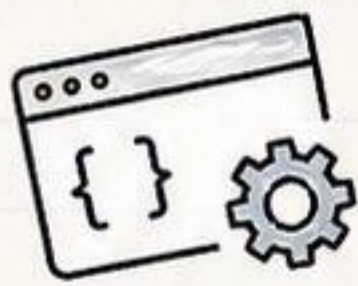
- JavaScript makes web pages interactive.
- Use meaningful variable and function names.
- DOM lets you manipulate HTML with JS.
- Practice more to master JavaScript!

Mini Project Idea 💡

Create a To-Do List App:

- Add tasks
- Mark as completed
- Delete tasks





4. RESPONSIVE WEB DESIGN

Responsive Web Design makes your website look great on all devices (mobile, tablet, laptop, desktop).

★ Media Queries

Use media queries to apply CSS based on device width.

```
/* Mobile (max width 600px) */
@media (max-width: 600px) {
  body { background-color: #f8f8f8; }
  .container { padding: 10px; }
}

/* Tablet (601px - 1024px) */
@media (min-width: 601px) and (max-width: 1024px) {
  .container { padding: 20px; }
}

/* Desktop (min width 1025px) */
@media (min-width: 1025px) {
  .container { padding: 30px; }
}
```

★ Common Breakpoints

Device	Screen Size	Breakpoint
 Mobile (Portrait)	Up to 600px	max-width: 600px
 Mobile (Landscape)	601px - 768px	max-width: 768px
 Tablet	769px - 1024px	max-width: 1024px
 Laptop	1025px - 1440px	max-width: 1440px
 Desktop	1441px and above	min-width: 1441px

★ Flexible Layouts

- Use relative units like %, em, rem, vw, vh.
- Use Flexbox and Grid for layouts.
- Use max-width for containers.

```
.container {
  width: 90%;          /* Responsive width */
  max-width: 1200px; /* Prevents too
                      wide content */
  margin: 0 auto;     /* Center align */
}
```

★ Images & Media

- Images should scale with the container.
- Use max-width: 100%.
- Use object-fit for better control.

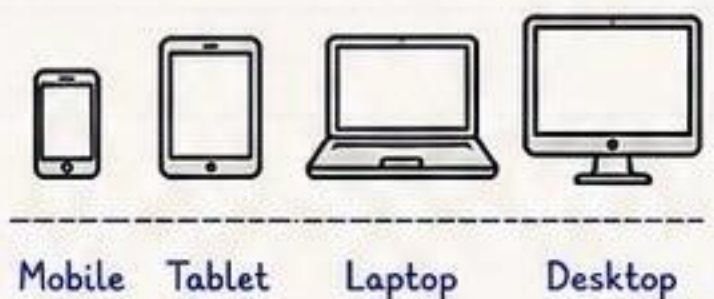
```
img {
  max-width: 100%;
  height: auto;
  display: block;
}

.cover {
  width: 100%;
  height: 100%;
  object-fit: cover;
}
```

★ Viewport Meta Tag

Add this in <head> to control layout on mobile browsers.

```
<meta name="viewport"
      content="width=device-width,
              initial-scale=1.0">
```



★ Mobile First Approach

- Design for mobile first, then scale up for larger screens.
- Write base styles for mobile.
- Use media queries for larger screens.

```
/* Base styles for mobile */
.navbar {
  flex-direction: column;
}

/* Styles for larger screens */
@media (min-width: 768px) {
  .navbar { flex-direction: row; }
}
```

★ Units for Responsive Design

Unit	Description
%	Relative to parent element
em	Relative to parent font-size
rem	Relative to root font-size
vw	1% of viewport width
vh	1% of viewport height
vmin	1% of smaller viewport dimension
vmax	1% of larger viewport dimension

★ Best Practices

- ✓ Use semantic HTML.
- ✓ Use Flexbox / Grid for layouts.
- ✓ Use relative units, not fixed pixels.
- ✓ Optimize images for all devices.
- ✓ Test on real devices.
- ✓ Keep it simple and user-friendly.

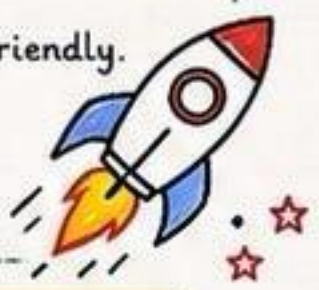


Remember :

- ✓ Responsive design = Better User Experience (UX)
- ✓ Media queries help in adapting to different devices.
- ✓ Always test your website on multiple screen sizes.
- ✓ Mobile first approach is the best practice.

Tools for Testing :

- Chrome DevTools (Toggle Device Toolbar)
- Responsive Design Checker
- Real Devices
- Online Tools (browserling.com, responsivetesttool.com)





5. BOOTSTRAP 5

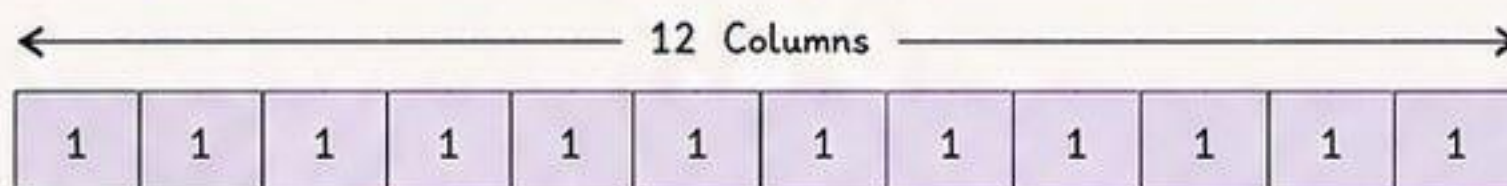
Bootstrap is a popular CSS framework that helps to build responsive and mobile-first websites quickly.

★ Why Use Bootstrap ?

- Responsive and mobile-first.
- Pre-built components and utilities.
- Saves time and ensures consistency.
- Easy customization.
- Cross-browser compatible.

★ Grid System

Bootstrap uses a 12-column responsive grid system.



Breakpoints:

xs <576px | sm ≥576px | md ≥768px | lg ≥992px | xl ≥1200px | xxl ≥1400px

★ Container

`.container` → Fixed width, changes at breakpoints

`.container-fluid` → Full width

★ Grid Classes

Class	Extra Small (<576px)	Small (≥576px)	Medium (≥768px)	Large (≥992px)	Extra Large (≥1200px)	XX Large (≥1400px)
<code>.col-*</code>	100%	Auto	Auto	Auto	Auto	Auto
<code>.col-sm-*</code>	-	100%	Auto	Auto	Auto	Auto
<code>.col-md-*</code>	-	-	100%	Auto	Auto	Auto
<code>.col-lg-*</code>	-	-	-	100%	Auto	Auto
<code>.col-xl-*</code>	-	-	-	-	100%	Auto
<code>.col-xxl-*</code>	-	-	-	-	-	100%

★ Buttons

Use `.btn` class with contextual classes.

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-info">Info</button>
<button class="btn btn-secondary">Secondary</button>
```

★ Navigation Bar

Use `.navbar` class.

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <a class="navbar-brand" href="#">Brand</a>
  <button class="navbar-toggler"
    type="button" data-bs-toggle="collapse"
    data-bs-target="#navMenu">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse"
    id="navMenu">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item"><a class="nav-link"
        href="#">Home</a></li>
    </ul>
  </div>
</nav>
```

★ Cards

```
<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Card Title</h5>
    <p class="card-text">Some quick example</p>
    <a href="#" class="btn btn-primary">
      Read More
    </a>
  </div>
</div>
```

★ Utility Classes (Examples)

`.text-center` → Center align text
`.mt-3` → Margin top (1rem)
`.p-3` → Padding (1rem)
`.bg-light` → Light background
`.d-flex` → Display flex
`.justify-content-between` → Space between
`.align-items-center` → Vertical center

★ Alerts

Use `.alert` class.

```
<div class="alert alert-success">Success!</div>
<div class="alert alert-info">Info!</div>
<div class="alert alert-warning">Warning!</div>
<div class="alert alert-danger">Danger!</div>
<div class="alert alert-secondary">Note!</div>
```

★ Forms

```
<form>
  <div class="mb-3">
    <label class="form-label">Name</label>
    <input type="text" class="form-control"
      placeholder="Enter name">
  </div>
  <button type="submit" class="btn btn-primary">
    Submit
  </button>
</form>
```



Remember :

- ✓ Bootstrap is mobile-first.
- ✓ Use containers and grid system for layout.
- ✓ Use utility classes to save time.
- ✓ Customize Bootstrap using your own CSS.

Example in Real Life :

Bootstrap is used in many real-world websites like Spotify, LinkedIn, and many admin dashboards for responsive design.





6. REACT JS

React is a JavaScript library for building user interfaces, especially single-page applications.

★ What is React ?

- Library for building UI.
- Component-based architecture.
- Uses Virtual DOM for performance.
- Declarative and efficient.
- Maintained by Meta (Facebook).

★ Key Features

- Component Based
- JSX Syntax
- Virtual DOM
- One-way Data Binding
- Reusable Components
- Hooks (useState, useEffect, etc.)

★ JSX (JavaScript XML)

JSX allows us to write HTML inside JavaScript.

```
const element = (  
  <h1 className="title">Hello React!</h1>  
);
```

★ Component (Functional)

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
export default Welcome;
```

Components must return a single parent element.

★ Using Component

```
import Welcome from "./Welcome";  
  
function App() {  
  return <Welcome name="Om" />;  
}
```

★ Props (Properties)

Props are used to pass data from one component to another.

```
function User(props) {  
  return <p>Name: {props.name}, Age: {props.age}</p>;  
}  
  
<User name="Om" age={22} />
```

★ State (useState Hook)

```
import { useState } from "react";  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
    </div>  
  );  
}
```

★ useEffect Hook

```
import { useEffect, useState } from "react";  
  
function Example() {  
  const [data, setData] = useState([]);  
  useEffect(() => {  
    fetch("https://api.example.com/data")  
      .then(res => res.json())  
      .then(setData);  
  }, []); // Runs only once on mount  
  
  return <p>Data Loaded!</p>;  
}
```

★ List Rendering

```
const items = ["HTML", "CSS", "JS"];  
  
return (  
  <ul>  
    {items.map((item, index) => (  
      <li key={index}>{item}</li>  
    ))}  
  </ul>  
);
```

Always use "key" in lists for better performance.

★ Conditional Rendering

```
function Greeting(props) {  
  return (  
    <div>  
      {props.isLoggedIn ? (  
        <p>Welcome Back!</p>  
      ) : (  
        <p>Please Login.</p>  
      )}  
    </div>  
  );  
}
```



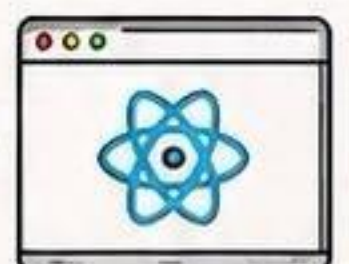
Remember :

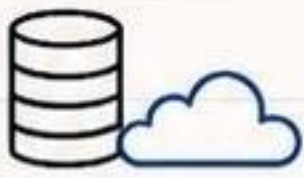
- ☑ Components start with a capital letter.
- ☑ JSX must return a single parent element.
- ☑ Use hooks inside functional components.
- ☑ Keep components small and reusable.
- ☑ Always add key in list rendering.

Example in Real Life :

React is used in real projects like:

- Facebook • Instagram • Netflix • Airbnb • Reddit
- React helps in building fast, scalable and interactive web applications.





7. NODE.JS & EXPRESS.JS

Node.js is a JavaScript runtime built on Chrome's V8 engine.

Express.js is a minimal and flexible Node.js web application framework.

★ What is Node.js ?

- JavaScript runtime environment.
- Event-driven, non-blocking I/O model.
- Built-in modules (fs, http, path, etc.)
- NPM - Node Package Manager.

★ Creating a Server (HTTP Module)

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js');
});
server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

★ Common Built-in Modules

Module	Description
fs	File system module
http	Create HTTP server
path	Utilities for file paths
url	URL utilities
os	Operating system info
events	Event handling
util	Utility functions

★ NPM Commands

npm init	→ Initialize a new project
npm install <package>	→ Install a package
npm install	→ Install from package.json
npm uninstall <package>	→ Uninstall a package
npm start	→ Run the project

★ What is Express.js ?

- Web framework for Node.js
- Simplifies routing and middleware
- Helps in building APIs quickly
- Minimal and unopinionated

★ Hello World in Express

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello Express!');
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

★ Routing in Express

```
app.get('/', (req, res) => res.send('Home Page'));
app.get('/about', (req, res) => res.send('About Page'));
app.post('/login', (req, res) => res.send('Login Page'));
app.put('/update', (req, res) => res.send('Update Page'));
app.delete('/delete', (req, res) => res.send('Delete Page'));
```

★ Route Parameters

```
app.get('/user/:id', (req, res) => {
  const id = req.params.id;
  res.send('User ID: ' + id);
});
```

Example URL
/user/123
↓
req.params.id = 123

★ Query Parameters

```
app.get('/search', (req, res) => {
  const q = req.query.q;
  res.send('Search: ' + q);
});
```

Example URL
/search?q=nodejs
↓
req.query.q = nodejs

★ Middleware

```
app.use((req, res, next) => {
  console.log('Request Time:', Date.now());
  next(); // Pass control to next middleware
});
```

Middleware is a function that has access to req, res, and next middleware.

★ Serving Static Files

```
app.use(express.static('public'));
```

Serves all files inside 'public' folder as static assets.

★ Body Parsing (JSON)

```
const express = require('express');
const app = express();
app.use(express.json()); // Parse JSON body

app.post('/api/data', (req, res) => {
  console.log(req.body); // Access JSON data
  res.send('Data received');
});
```

Allows us to read JSON data from request body.

★ Common Response Methods

Method	Description
res.send()	Send response of any type
res.json()	Send JSON response
res.status()	Set status code
res.redirect()	Redirect to another URL



Remember :

- ☑ Node.js runs JS on the server.
- ☑ NPM helps in managing packages.
- ☑ Express.js makes routing & APIs easy.
- ☑ Middleware = Functions that run before the route.

Example in Real Life :

Used in real-world applications like:



Spotify



Netflix



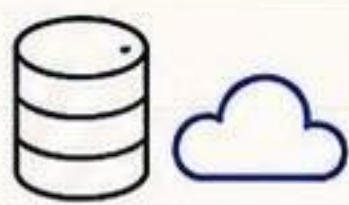
LinkedIn



PayPal



Uber



8. MONGODB

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents.

★ Key Concepts

- Database → Container for collections
- Collection → Group of documents
- Document → JSON-like data
- Field → Key-value pair in document
- ObjectId → Unique ID for documents

★ Basic Commands

Command	Description
<code>show dbs</code>	Show all databases
<code>use <db_name></code>	Switch to a database
<code>show collections</code>	Show all collections
<code>db</code>	Show current database
<code>db.dropDatabase()</code>	Delete current database

★ CRUD Operations

Collection: `users`

- Create → `db.users.insertOne({...})`
- Read → `db.users.find({...})`
- Update → `db.users.updateOne(filter, {...})`
- Delete → `db.users.deleteOne({...})`

★ Insert Documents

```
// Insert One Document
db.users.insertOne({
  name: "Om",
  age: 22,
  city: "Nagpur"
});

// Insert Many Documents
db.users.insertMany([
  { name: "Aman", age: 21 },
  { name: "Riya", age: 20 }
]);
```

★ Find Documents

```
// Find All
db.users.find();

// Find with Condition
db.users.find({ age: { $gt: 20 } });

// Find One
db.users.findOne({ name: "Om" });

// Select Specific Fields
db.users.find({}, { name: 1, age: 1, _id: 0 });

// Sort Results
db.users.find().sort({ age: -1 });
```

★ Update Documents

```
// Update One
db.users.updateOne(
  { name: "Om" },
  { $set: { age: 23 } }
);

// Update Many
db.users.updateMany(
  { city: "Nagpur" },
  { $set: { active: true } }
);
```

★ Delete Documents

```
// Delete One
db.users.deleteOne({ name: "Riya" });

// Delete Many
db.users.deleteMany({ age: { $lt: 20 } });

// Delete All Documents
db.users.deleteMany({});
```

★ Operators

Operator	Description
<code>\$eq</code>	Equal
<code>\$ne</code>	Not Equal
<code>\$gt</code>	Greater Than
<code>\$gte</code>	Greater Than or Equal
<code>\$lt</code>	Less Than
<code>\$lte</code>	Less Than or Equal
<code>\$in</code>	In Array
<code>\$nin</code>	Not In Array
<code>\$and</code>	Logical AND
<code>\$or</code>	Logical OR

★ Logical Operators (Examples)

```
// $and
db.users.find({ $and: [ { age: { $gt: 20 } },
  { city: "Nagpur" } ] });

// $or
db.users.find({ $or: [ { age: { $lt: 20 } },
  { city: "Pune" } ] });

// $in
db.users.find({ city: { $in: ["Nagpur", "Pune"] } });
```

★ Aggregation (Basic)

```
db.orders.aggregate([
  { $match: { status: "Completed" } },
  { $group: { _id: "$userId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
]);
```

★ Data Types

- String → "Hello"
- Integer → 42
- Double → 3.14
- Boolean → true / false
- Array → [1, 2, 3]
- Object → { key: "value" }
- Date → `ISODate("2024-01-01T00:00:00Z")`
- ObjectId → Automatically generated unique id

Example Document

```
{
  "_id": ObjectId("64f8a1b2c3d4e5f6g7h8i9j"),
  "name": "Om",
  "age": 22,
  "city": "Nagpur",
  "skills": ["JavaScript", "Node.js"],
  "active": true,
  "createdAt": ISODate("2024-05-12T10:30:00Z")
}
```



Remember :

- ✓ MongoDB stores data in JSON-like documents.
- ✓ Use `find()` to read data and queries.
- ✓ Always use proper indexes in production.
- ✓ Keep your schema flexible but consistent.

Example in Real Life :

MongoDB is used in many real-world apps like:



Pinterest



Instagram



Uber



E-commerce



Gaming



Chat Apps

9. APIs, GIT & DEPLOYMENT

APIs connect your app with the world. Git helps in version control.

Deployment makes your project live for everyone.

★ REST API Basics

- REST = Representational State Transfer
- Uses HTTP methods.
- Data in JSON format.
- Stateless communication.
- Common methods:
 - GET (Read)
 - POST (Create)
 - PUT / PATCH (Update)
 - DELETE (Delete)

★ Example API Call (Fetch)

```
fetch('https://api.example.com/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log(err));
```

Always handle errors using .catch()

★ JSON Example

```
{
  "id": 1,
  "name": "Om",
  "email": "om@example.com",
  "skills": ["HTML", "CSS", "JS"],
  "active": true
}
```

JSON = Lightweight data format

★ Git Basics

- Git is a distributed version control system.
- Tracks changes in code.
- Works locally and in cloud (GitHub, GitLab).



★ Important Git Commands

Command	Description
git init	Initialize new repo
git clone <url>	Clone a repository
git status	Check status
git add .	Stage all changes
git commit -m "msg"	Commit changes
git push origin main	Push to remote repo
git pull origin main	Pull latest changes
git branch	List all branches
git checkout <branch>	Switch branch
git merge <branch>	Merge branches

★ Git Workflow

1. Modify Files
2. git add .
3. git commit -m "msg"
4. git push origin main

Commit small changes regularly with meaningful messages.

★ Deployment Options

Frontend (Static)	Backend (API)	Database	Full Stack
• Netlify • Vercel • GitHub Pages	• Render • Railway • Heroku	• MongoDB Atlas (NoSQL) • PlanetScale (MySQL) • Supabase (PostgreSQL)	• AWS • DigitalOcean • Google Cloud

★ Deployment Steps (General)

1. Build your project (if required)
2. Push code to GitHub
3. Connect repo to hosting platform
4. Set environment variables
5. Deploy 🚀
6. Share your live URL 🌐

★ Project Folder Structure (MERN Stack Example)

```
my-fullstack-app/
├── frontend/ (React)
│   ├── public/
│   ├── src/
│   └── package.json
├── backend/ (Node.js + Express)
│   ├── config/
│   ├── models/
│   ├── routes/
│   ├── controllers/
│   ├── server.js
│   ├── .env
│   └── package.json
└── README.md
```

Keep frontend and backend separate for better structure.



★ Security Best Practices

- ✓ Always use HTTPS
- ✓ Store secrets in .env file
- ✓ Never expose API keys
- ✓ Validate user input
- ✓ Use JWT for authentication
- ✓ Keep dependencies updated



Remember :

- ✓ APIs connect frontend with backend.
- ✓ Git helps you track and manage code.
- ✓ Deployment makes your project live.

Example in Real Life :

Instagram → APIs for data
GitHub → Git for collaboration

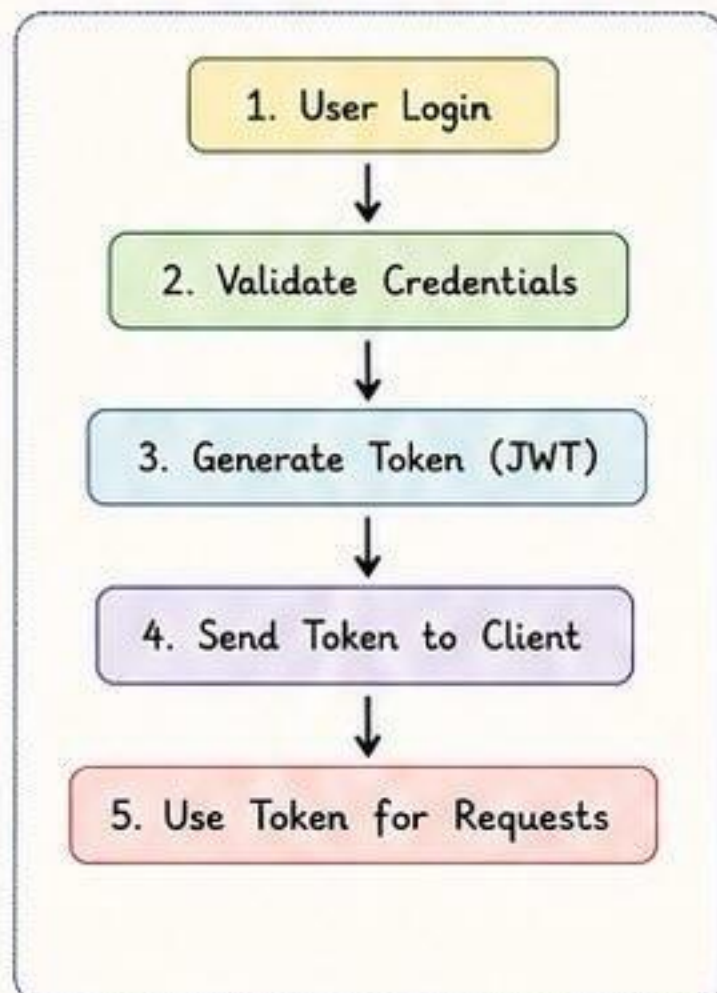




10. AUTHENTICATION & AUTHORIZATION

Authentication verifies who you are. Authorization checks what you can do.

★ Authentication Flow



★ JWT (JSON Web Token)

- Compact, URL-safe token
- Contains claims about user
- Signed using secret key
- Consists of 3 parts:

Header.Payload.Signature

```
Header = { "alg": "HS256", "typ": "JWT" }
Payload = { "id": 1, "name": "Om", "role": "user" }
Signature = HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload), secret)
```

★ Protecting Routes (Express.js)

```
// Middleware to verify JWT
const jwt = require('jsonwebtoken');
function authenticate(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) return res.status(401).json({
    message: 'Access Denied' });
  try {
    const decoded = jwt.verify(token, 'secretkey');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(403).json({ message: 'Invalid Token' });
  }
}
```

Use this middleware on protected routes.

★ Role Based Access Control (RBAC)

Role	Permissions
Admin	Create, Read, Update, Delete
Editor	Create, Read, Update
User	Read
Guest	Read (Limited)



Always assign minimum required permissions to roles.

★ Password Best Practices

- ✓ Use strong passwords
- ✓ Hash passwords (bcrypt)
- ✓ Never store plain passwords
- ✓ Add salt for extra security

```
const bcrypt = require('bcrypt');
// Hash Password
const hash = await bcrypt.hash(password, 10);
// Compare Password
const match = await bcrypt.compare(
  password, hash);
```

★ Authorization Example

```
function authorize(roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        message: "Not Allowed"
      });
    }
    next();
  }
}
```

Use after authentication middleware.
Example: `app.get('/admin', authenticate, authorize(['admin']));`

★ OAuth 2.0 (Social Login)

Allows users to login using:



Google



GitHub



Facebook

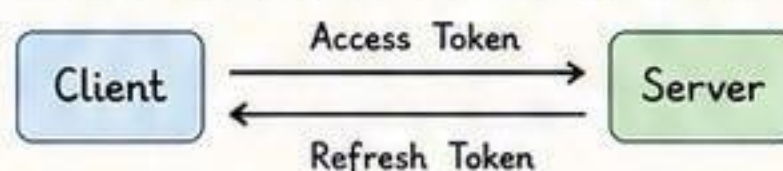


Twitter

Flow: User → Provider → Your App

★ Refresh Token Flow

1. User logs in → Access Token + Refresh Token
2. Access Token expires
3. Use Refresh Token to get new Access Token
4. Keep Refresh Token secure (HttpOnly Cookie)



★ Common Status Codes

200	OK
401	Unauthorized (Login Required)
403	Forbidden (Not Allowed)
404	Not Found
500	Internal Server Error

★ Security Tips



Always use HTTPS



Store secrets in .env file



Validate & sanitize user input



Limit login attempts



Use HttpOnly cookies



Keep dependencies updated



Remember :

- ✓ Authenticate first, then authorize.
- ✓ Never trust user input.
- ✓ Tokens make apps stateless & scalable.
- ✓ Security is an important part of development.

Example in Real Life :

Instagram → Login & See Your Posts
YouTube → Login & Upload Videos
Amazon → Login & Place Orders
Netflix → Secure Access to Accounts

